

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Домашнее задание

(по дисциплине «Методы оптимизации»)

Выполнили:

Лабин Макар Андреевич,
МОПТ 3.3, Р3231, 466449;
Ожеховский Александр Сергеевич,
МОПТ 3.3, Р3231, 466944.

Проверил:

Запорожцев Иван Федорович



г. Санкт-Петербург, Россия
2026

Содержание

1	Блок №1	1
1.1	Задание	1
1.2	Применимость метода	1
1.3	Первые итерации	2
1.3.1	Итерация №1	2
1.3.2	Итерация №2	3
1.3.3	Итерация №3	4
1.4	История приближений	5
2	Блок №2	5
2.1	Задание	5
2.2	Аналитическое решение	6
2.3	Применимость метода	7
2.4	Первые итерации	8
2.4.1	Итерация №1	8
2.4.2	Итерация №2	9
2.4.3	Итерация №3	10
2.5	История приближений	11
2.6	Объектная модель	12
2.6.1	Постановка задачи	12
2.6.2	Диаграмма классов	12
3	Блок №3	14
3.1	Задание	14
3.2	Описание набора данных	15
3.2.1	Целевой признак	15
3.2.2	Структура и состав признаков	15
3.2.3	Разведочный анализ данных	16
3.3	Результаты алгоритмов отбора признаков	17
4	Источники литературы	18

1 Блок №1

1.1 Задание

1. Для функции своего варианта выполните решение задачи одномерной минимизации методом кубической аппроксимации. Напишите программу на одном из языков программирования, генерирующую последовательность приближений. В качестве критерия останова используйте точность $\varepsilon = 0.0001$.
2. Реализуйте программно визуализацию исходной целевой функции и аппроксимирующей функции в одной координатной плоскости. Постройте графики для нескольких последовательных итераций алгоритма для демонстрации процесса сходимости.

Рекомендуется программная реализация на языке Python.

Исходные данные для варианта №3:

$$f(x) = \frac{1}{4}x^4 + x^2 - 8x + 12, \quad x \in [0, 2]$$

1.2 Применимость метода

Метод кубической интерполяции Эрмита применим для поиска локального минимума функции $f(x)$ на итерации (i) при выполнении следующих условий:

1. Функция $f(x)$ должна быть непрерывно дифференцируема на исследуемом интервале, чтобы значения производных $f'(x_0^{(i)})$ и $f'(x_1^{(i)})$ и знаки в окрестности были определены корректно.
2. На концах отрезка $[x_0^{(i)}, x_1^{(i)}]$ знаки производных должны быть:

$$f'(x_0^{(i)}) < 0, \quad f'(x_1^{(i)}) > 0$$

Чтобы не дублировать формулы на каждой последующей итерации, выпишем общие аналитические выражения. Для автоматизации вычислений составим биекцию исходного отрезка $[x_0^{(i)}, x_1^{(i)}]$ с отрезком $[0, 1]$ заменой переменной:

$$t = \frac{x - x_0^{(i)}}{h^{(i)}}, \quad \text{где } h^{(i)} = x_1^{(i)} - x_0^{(i)}$$

Каноническая форма интерполяционного многочлена Эрмита:

$$H_3(t) = f(x_0^{(i)})(1-3t^2+2t^3) + f(x_1^{(i)})(3t^2-2t^3) + h^{(i)}f'(x_0^{(i)})(t-2t^2+t^3) + h^{(i)}f'(x_1^{(i)})(-t^2+t^3)$$

Для нахождения стационарных точек и проверки достаточного условия экстремума используются первая и вторая производные многочлена:

$$\frac{dH_3}{dt} = f(x_0^{(i)})(6t^2 - 6t) + f(x_1^{(i)})(6t - 6t^2) + h^{(i)}f'(x_0^{(i)})(3t^2 - 4t + 1) + h^{(i)}f'(x_1^{(i)})(3t^2 - 2t)$$

$$\frac{d^2H_3}{dt^2} = f(x_0^{(i)})(12t - 6) + f(x_1^{(i)})(6 - 12t) + h^{(i)}f'(x_0^{(i)})(6t - 4) + h^{(i)}f'(x_1^{(i)})(6t - 2)$$

1.3 Первые итерации

1.3.1 Итерация №1

Построим каноническую форму кубического интерполяционного многочлена Эрмита на отрезке:

$$[x_0^{(1)}, x_1^{(1)}] = [0, 2] \implies h^{(1)} = x_1^{(1)} - x_0^{(1)} = 2$$

Известны значения функции и её производных в узлах:

$$f(x_0^{(1)}) = 12, \quad f'(x_1^{(1)}) = 4$$

$$f'(x_0^{(1)}) = -8, \quad f'(x_1^{(1)}) = 4$$

Для поиска точки минимума воспользуемся необходимым условием существования экстремума и найдём стационарные точки:

$$\frac{dH_3}{dt} = 0 \implies \begin{cases} t_1 = -0.81650 \\ t_2 = 0.81650 \end{cases}$$

Проверим достаточное условие экстремума:

$$\begin{cases} \frac{d^2 H_3}{dt^2}(t_1) = -39.19184 < 0 \implies \text{локальный максимум} \\ \frac{d^2 H_3}{dt^2}(t_2) = 39.19184 > 0 \implies \text{локальный минимум} \end{cases} \implies t^* = 0.81650$$

Выполним обратную замену для перехода к исходной переменной x :

$$x^{*(1)} = x_0^{(1)} + h^{(1)} \cdot t^* = 1.63299$$

Вычислим истинное значение производной исходной функции в точке $x^{*(1)}$:

$$f'(x^{*(1)}) = -0.37937 < 0$$

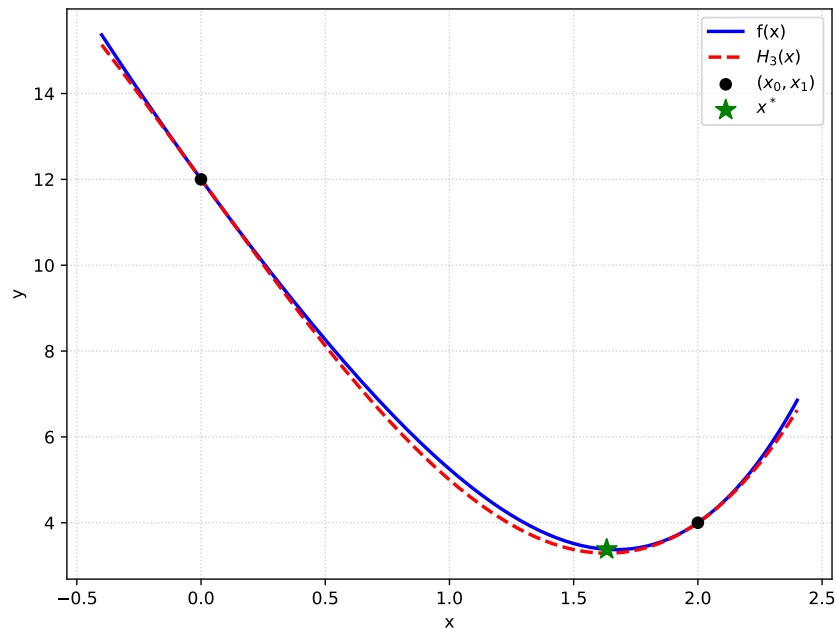


Рис. 1: Исходная и аппроксимирующая функции на итерации №1.

Сравним полученную величину с заданным параметром точности ε :

$$|f'(x^{*(1)})| = 0.37937 \geq 0.0001 \implies \text{продолжаем итерации}$$

Новые границы отрезка для следующей итерации принимают значения:

$$\begin{cases} x_0^{(2)} = 1.63299 \\ x_1^{(2)} = 2 \end{cases}$$

1.3.2 Итерация №2

Построим каноническую форму кубического интерполяционного многочлена Эрмита на отрезке:

$$[x_0^{(2)}, x_1^{(2)}] = [1.63299, 2] \implies h^{(2)} = x_1^{(2)} - x_0^{(2)} = 0.36701$$

Известны значения функции и её производных в узлах:

$$f(x_0^{(2)}) = 3.38050, \quad f(x_1^{(2)}) = 4$$

$$f'(x_0^{(2)}) = -0.37937, \quad f'(x_1^{(2)}) = 4$$

Для поиска точки минимума воспользуемся необходимым условием существования экстремума и найдём стационарные точки:

$$\frac{dH_3}{dt} = 0 \implies \begin{cases} t_1 = -5.06830 \\ t_2 = 0.10197 \end{cases}$$

Проверим достаточное условие экстремума:

$$\begin{cases} \frac{d^2 H_3}{dt^2}(t_1) = -1.39281 < 0 \implies \text{локальный максимум} \\ \frac{d^2 H_3}{dt^2}(t_2) = 1.39281 > 0 \implies \text{локальный минимум} \end{cases} \implies t^* = 0.10197$$

Выполним обратную замену для перехода к исходной переменной x :

$$x^{*(2)} = x_0^{(2)} + h^{(2)} \cdot t^* = 1.67042$$

Вычислим истинное значение производной исходной функции в точке $x^{*(2)}$:

$$f'(x^{*(2)}) = 0.00180 > 0$$

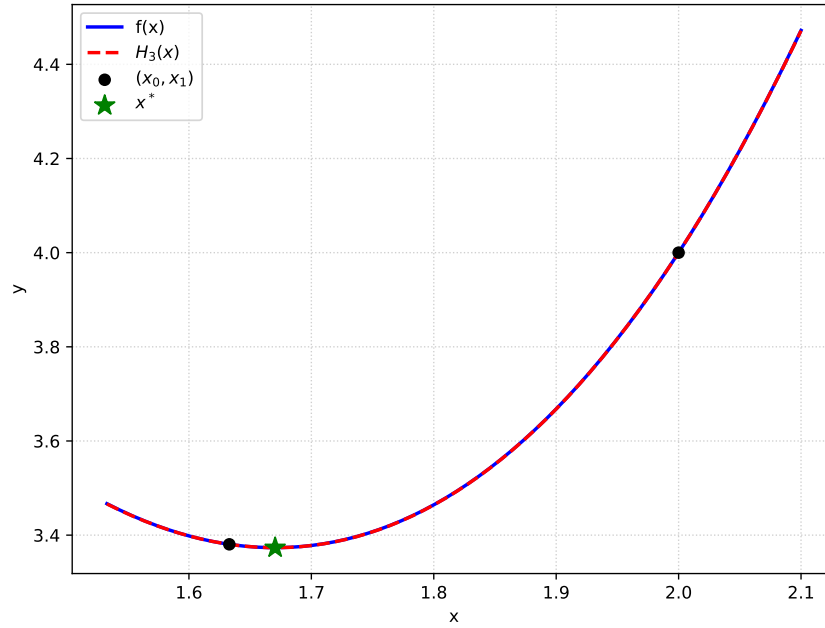


Рис. 2: Исходная и аппроксимирующая функции на итерации №2.

Сравним полученную величину с заданным параметром точности ε :

$$|f'(x^{*(2)})| = 0.00180 \geq 0.0001 \implies \text{продолжаем итерации}$$

Новые границы отрезка для следующей итерации принимают значения:

$$\begin{cases} x_0^{(3)} = 1.63299 \\ x_1^{(3)} = 1.67042 \end{cases}$$

1.3.3 Итерация №3

Построим каноническую форму кубического интерполяционного многочлена Эрмита на отрезке:

$$[x_0^{(3)}, x_1^{(3)}] = [1.63299, 1.67042] \implies h^{(3)} = x_1^{(3)} - x_0^{(3)} = 0.03743$$

Известны значения функции и её производных в узлах:

$$\begin{aligned} f(x_0^{(3)}) &= 3.38050, & f(x_1^{(3)}) &= 3.37339 \\ f'(x_0^{(3)}) &= -0.37937, & f'(x_1^{(3)}) &= 0.00180 \end{aligned}$$

Для поиска точки минимума воспользуемся необходимым условием существования экстремума и найдём стационарные точки:

$$\frac{dH_3}{dt} = 0 \implies \begin{cases} t_1 = -54.91543 \\ t_2 = 0.99536 \end{cases}$$

Проверим достаточное условие экстремума:

$$\begin{aligned} \left[\begin{aligned} \frac{d^2 H_3}{dt^2}(t_1) &= -0.01452 < 0 \implies \text{локальный максимум} \\ \frac{d^2 H_3}{dt^2}(t_2) &= 0.01452 > 0 \implies \text{локальный минимум} \end{aligned} \right. & \implies t^* = 0.99536 \end{aligned}$$

Выполним обратную замену для перехода к исходной переменной x :

$$x^{*(3)} = x_0^{(3)} + h^{(3)} \cdot t^* = 1.67024$$

Вычислим истинное значение производной исходной функции в точке $x^{*(3)}$:

$$f'(x^{*(3)}) = -0.00000 < 0$$

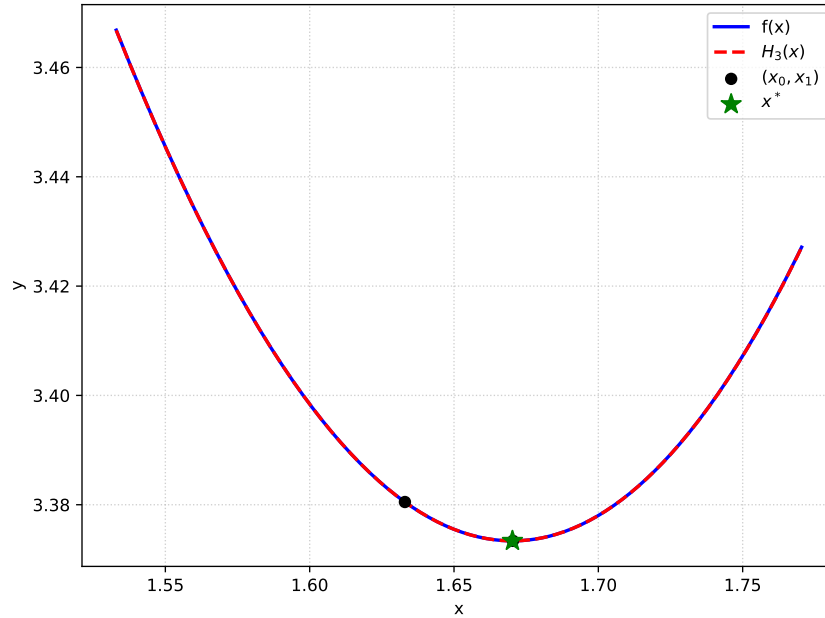


Рис. 3: Исходная и аппроксимирующая функции на итерации №3.

Сравним полученную величину с заданным параметром точности ε :

$$|f'(x^{*(3)})| = 0.00000 < 0.0001 \implies \text{условие останова выполнено}$$

1.4 История приближений

В таблице 1 приведена сводная история последовательных приближений метода кубической интерполяции Эрмита по итерациям.

i	$x_0^{(i)}$	$x_1^{(i)}$	$h^{(i)}$	t^*	$x^{*(i)}$	$f(x^{*(i)})$	$f'(x^{*(i)})$
1	0	2	2	0.81650	1.63299	3.38050	-0.37937
2	1.63299	2	0.36701	0.10197	1.67042	3.37339	0.00180
3	1.63299	1.67042	0.03743	0.99536	1.67024	3.37339	-0.00000

Таблица 1: Итерации методом кубической интерполяции Эрмита.

2 Блок №2

2.1 Задание

1. Напишите программу для визуализации линий уровня целевой функции и ломаной последовательных приближений. Приближения итеративно

порождаются с помощью методов оптимизации из ЛР4 до выполнения критерия останова.

Модифицируйте алгоритм отрисовки контуров так, чтобы линии уровня целевой функции проходили *строго через вершины* ломаной приближений (соответственно, количество линий уровня должно быть равно количеству итераций/приближений). Постройте такую визуализацию для каждого метода оптимизации из ЛР4.

2. Найдите точку минимума функции своего варианта численно **методом Ньютона**. В качестве критерия останова используйте порог на модуль градиента функции: $\|\nabla z\| \leq 0.0001$.

Выполните решение также *аналитически* — найдите градиент, приравняйте его к нулю, определите все стационарные точки и с помощью матрицы Гессе укажите тип экстремума для каждой из них. Представьте в отчете подробную детализацию вычислений, а также реализуйте эти расчёты программно.

3. Постройте **объектную модель** задачи минимизации функции двух переменных для трех методов из ЛР4: покоординатного спуска, градиентного спуска и метода наискорейшего спуска.

Выполните постановку задачи, выделите ключевые сущности (классы), опишите их атрибуты и методы. Спроектируйте иерархию классов (используя абстрактные классы и/или интерфейсы), которая объединяет логику всех трех методов. Воспользуйтесь объектными отношениями (наследование, реализация, композиция, агрегация или зависимость) по своему выбору. Обновите программную реализацию на языке Python с учетом разработанной объектной модели.

Исходные данные для варианта №3:

$$z(x, y) = x^3 - 9x^2 + 24x + y^3 - 14y^2 + 64y - 116, \quad M_0 = (3.5, 5.5)$$

2.2 Аналитическое решение

Рассмотрим целевую функцию двух переменных $z(x, y)$. Для нахождения точек экстремума вычислим её градиент:

$$\nabla z(x, y) = \begin{pmatrix} \frac{\partial z}{\partial x} \\ \frac{\partial z}{\partial y} \end{pmatrix} = \begin{pmatrix} 3x^2 - 18x + 24 \\ 3y^2 - 28y + 64 \end{pmatrix}$$

Приравняем его к нулевому вектору, чтобы найти стационарные точки:

$$\begin{cases} \frac{\partial z}{\partial x} = 3x^2 - 18x + 24 = 0 \\ \frac{\partial z}{\partial y} = 3y^2 - 28y + 64 = 0 \end{cases}$$

Из полученной системы имеем четыре стационарные точки:

$$M_1(2, 4), \quad M_2\left(2, \frac{16}{3}\right), \quad M_3(4, 4), \quad M_4\left(4, \frac{16}{3}\right)$$

Вычислим вторые частные производные:

$$\frac{\partial^2 z}{\partial x^2} = 6x - 18, \quad \frac{\partial^2 z}{\partial y^2} = 6y - 28, \quad \frac{\partial^2 z}{\partial x \partial y} = \frac{\partial^2 z}{\partial y \partial x} = 0$$

Тогда матрица Гессе для проверки точек на экстремум принимает вид:

$$H(x, y) = \begin{pmatrix} 6x - 18 & 0 \\ 0 & 6y - 28 \end{pmatrix}$$

Для каждой из четырёх найденных стационарных точек проверим выполнение критерия Сильвестра:

1. Для точки $M_1(2, 4)$:

$$\Delta_1 = -6 < 0, \quad \Delta_2 = 24 > 0$$

Матрица Гессе отрицательно определена $\implies M_1$ является точкой **строгого локального максимума**.

2. Для точки $M_2(2, \frac{16}{3})$:

$$\Delta_1 = -6 < 0, \quad \Delta_2 = -24 < 0$$

Матрица Гессе знаконеопределена $\implies M_2$ является **седловой точкой**.

3. Для точки $M_3(4, 4)$:

$$\Delta_1 = 6 > 0, \quad \Delta_2 = -24 < 0$$

Матрица Гессе знаконеопределена $\implies M_3$ является **седловой точкой**.

4. Для точки $M_4(4, \frac{16}{3})$:

$$\Delta_1 = 6 > 0, \quad \Delta_2 = 24 > 0$$

Матрица Гессе положительно определена $\implies M_4$ является точкой **строгого локального минимума**.

То есть целевая функция обладает глобальным минимумом в точке M_4 :

$$f(M_4) = f\left(4, \frac{16}{3}\right) \approx -5.18519$$

2.3 Применимость метода

Метод Ньютона относится к методам второго порядка и использует квадратичную аппроксимацию функции в окрестности текущей точки. Метод обладает квадратичной скоростью сходимости вблизи точки минимума при выполнении следующих условий:

1. Функция $z(x, y)$ дважды непрерывно дифференцируема в рассматриваемой области.
2. Матрица Гессе H непрерывна и положительно определена в окрестности оптимума: это гарантирует шаг в сторону убывания функции.

2.4 Первые итерации

2.4.1 Итерация №1

Текущее приближение вектора переменных:

$$\mathbf{x}^{(0)} = \begin{pmatrix} 3.50000 \\ 5.50000 \end{pmatrix}$$

Вычислим вектор градиента в текущей точке:

$$\nabla z(\mathbf{x}^{(0)}) = \begin{pmatrix} -2.25000 \\ 0.75000 \end{pmatrix}$$

Определим евклидову норму градиента:

$$\|\nabla z(\mathbf{x}^{(0)})\|_2 = \sqrt{(-2.25000)^2 + (0.75000)^2} = 2.37171$$

Проверим условие останова:

$$\|\nabla z\| = 2.37171 > 0.0001 \implies \text{условие не выполнено, продолжаем расчёт.}$$

Вычислим компоненты матрицы Гессе в текущей точке:

$$H(\mathbf{x}^{(0)}) = \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix}$$

Найдём приращение вектора переменных $\Delta \mathbf{x}^{(1)}$, решая систему линейных уравнений $H \cdot \Delta \mathbf{x} = -\nabla z$:

$$\begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -2.25000 \\ -0.75000 \end{pmatrix} \implies \Delta \mathbf{x}^{(1)} = \begin{pmatrix} 0.75000 \\ -0.15000 \end{pmatrix}$$

Вычислим координаты новой точки:

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \Delta \mathbf{x}^{(1)} = \begin{pmatrix} 3.50000 \\ 5.50000 \end{pmatrix} + \begin{pmatrix} 0.75000 \\ -0.15000 \end{pmatrix} = \begin{pmatrix} 4.25000 \\ 5.35000 \end{pmatrix}$$

Значение целевой функции в новой точке составляет:

$$z(\mathbf{x}^{(1)}) = -4.98150$$

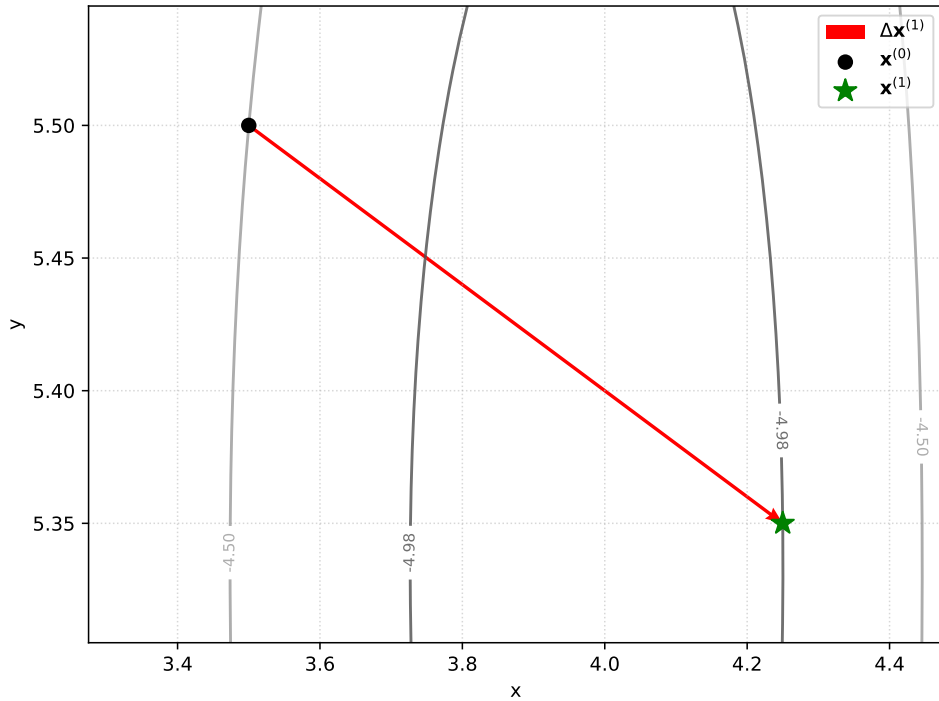


Рис. 4: Положение точки на линиях уровня для итерации №1.

2.4.2 Итерация №2

Текущее приближение вектора переменных:

$$\mathbf{x}^{(1)} = \begin{pmatrix} 4.25000 \\ 5.35000 \end{pmatrix}$$

Вычислим вектор градиента в текущей точке:

$$\nabla z(\mathbf{x}^{(1)}) = \begin{pmatrix} 1.68750 \\ 0.06750 \end{pmatrix}$$

Определим евклидову норму градиента:

$$\|\nabla z(\mathbf{x}^{(1)})\|_2 = \sqrt{(1.68750)^2 + (0.06750)^2} = 1.68885$$

Проверим условие останова:

$$\|\nabla z\| = 1.68885 > 0.0001 \implies \text{условие не выполнено, продолжаем расчёт.}$$

Вычислим компоненты матрицы Гессе в текущей точке:

$$H(\mathbf{x}^{(1)}) = \begin{pmatrix} 7.50000 & 0 \\ 0 & 4.10000 \end{pmatrix}$$

Найдём приращение вектора переменных $\Delta \mathbf{x}^{(2)}$, решая систему линейных уравнений $H \cdot \Delta \mathbf{x} = -\nabla z$:

$$\begin{pmatrix} 7.50000 & 0 \\ 0 & 4.10000 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -1.68750 \\ -0.06750 \end{pmatrix} \implies \Delta \mathbf{x}^{(2)} = \begin{pmatrix} -0.22500 \\ -0.01646 \end{pmatrix}$$

Вычислим координаты новой точки:

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \Delta \mathbf{x}^{(2)} = \begin{pmatrix} 4.25000 \\ 5.35000 \end{pmatrix} + \begin{pmatrix} -0.22500 \\ -0.01646 \end{pmatrix} = \begin{pmatrix} 4.02500 \\ 5.33354 \end{pmatrix}$$

Значение целевой функции в новой точке составляет:

$$z(\mathbf{x}^{(2)}) = -5.18329$$

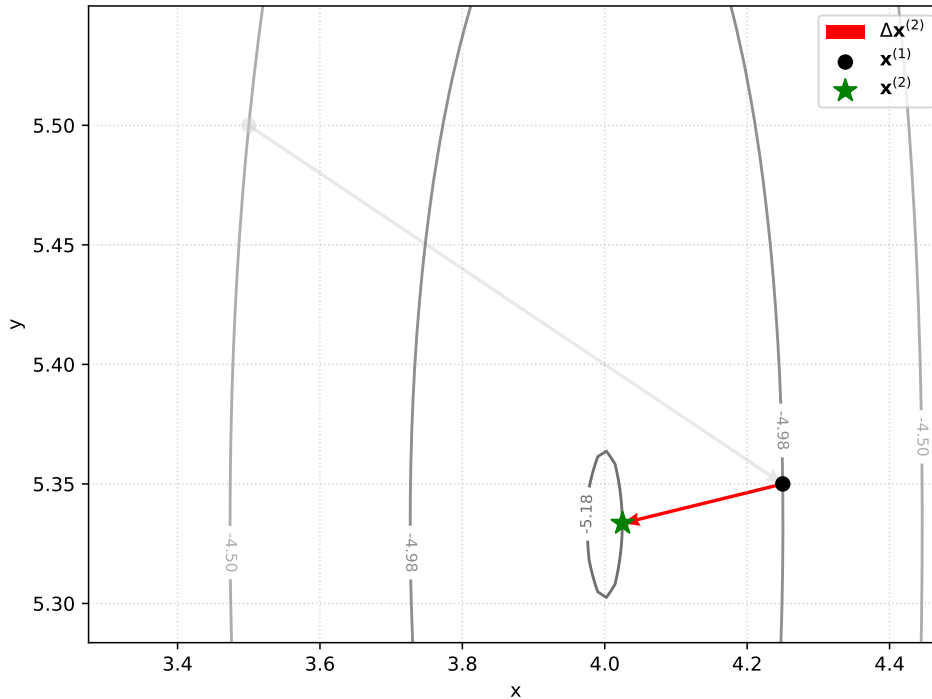


Рис. 5: Положение точки на линиях уровня для итерации №2.

2.4.3 Итерация №3

Текущее приближение вектора переменных:

$$\mathbf{x}^{(2)} = \begin{pmatrix} 4.02500 \\ 5.33354 \end{pmatrix}$$

Вычислим вектор градиента в текущей точке:

$$\nabla z(\mathbf{x}^{(2)}) = \begin{pmatrix} 0.15188 \\ 0.00081 \end{pmatrix}$$

Определим евклидову норму градиента:

$$\|\nabla z(\mathbf{x}^{(2)})\|_2 = \sqrt{(0.15188)^2 + (0.00081)^2} = 0.15188$$

Проверим условие останова:

$$\|\nabla z\| = 0.15188 > 0.0001 \implies \text{условие не выполнено, продолжаем расчёт.}$$

Вычислим компоненты матрицы Гессе в текущей точке:

$$H(\mathbf{x}^{(2)}) = \begin{pmatrix} 6.15000 & 0 \\ 0 & 4.00122 \end{pmatrix}$$

Найдём приращение вектора переменных $\Delta \mathbf{x}^{(3)}$, решая систему линейных уравнений $H \cdot \Delta \mathbf{x} = -\nabla z$:

$$\begin{pmatrix} 6.15000 & 0 \\ 0 & 4.00122 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -0.15188 \\ -0.00081 \end{pmatrix} \Rightarrow \Delta \mathbf{x}^{(3)} = \begin{pmatrix} -0.02470 \\ -0.00020 \end{pmatrix}$$

Вычислим координаты новой точки:

$$\mathbf{x}^{(3)} = \mathbf{x}^{(2)} + \Delta \mathbf{x}^{(3)} = \begin{pmatrix} 4.02500 \\ 5.33354 \end{pmatrix} + \begin{pmatrix} -0.02470 \\ -0.00020 \end{pmatrix} = \begin{pmatrix} 4.00030 \\ 5.33333 \end{pmatrix}$$

Значение целевой функции в новой точке составляет:

$$z(\mathbf{x}^{(3)}) = -5.18518$$

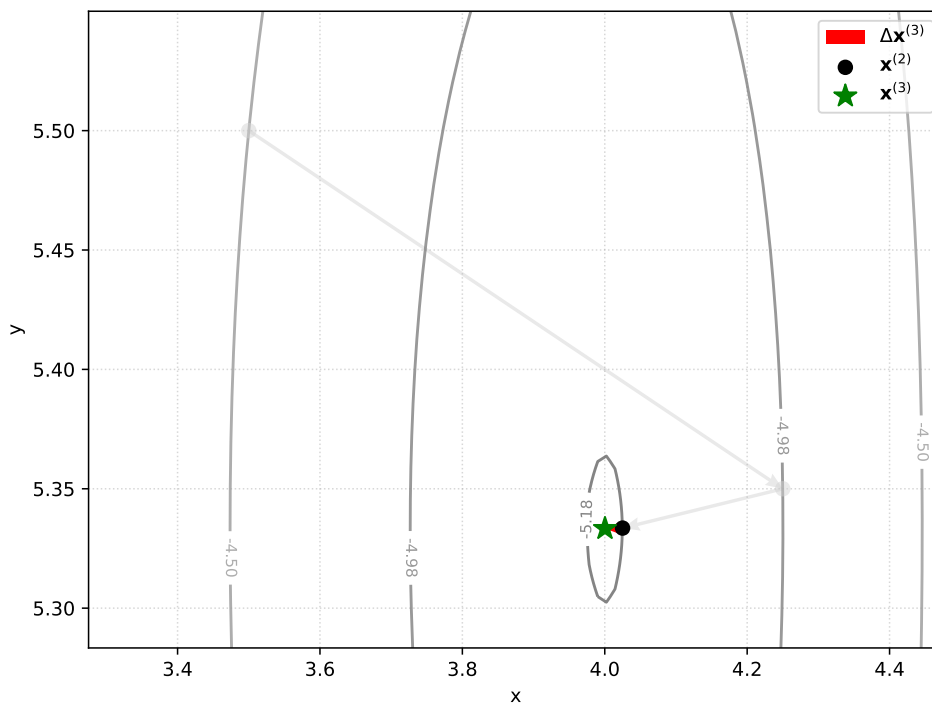


Рис. 6: Положение точки на линиях уровня для итерации №3.

2.5 История приближений

В таблице 2 приведена сводная история последовательных приближений метода Ньютона по итерациям.

i	$x^{(i-1)}$	$y^{(i-1)}$	$z(\mathbf{x}^{(i-1)})$	$\ \nabla z\ $	$\Delta x^{(i)}$	$\Delta y^{(i)}$
1	3.50000	5.50000	-4.98150	2.37171	0.75000	-0.15000
2	4.25000	5.35000	-5.18329	1.68885	-0.22500	-0.01646
3	4.02500	5.33354	-5.18518	0.15188	-0.02470	-0.00020
4	4.00030	5.33333	-5.18519	0.00183	-0.00030	-0.00000
5	4.00000	5.33333	-5.18519	0.00000	-0.00000	0

Таблица 2: Сводная история итераций метода Ньютона.

2.6 Объектная модель

2.6.1 Постановка задачи

Задача заключается в программной реализации и проектировании по принципам объектно-ориентированного программирования трёх методов многомерной оптимизации: покоординатного спуска, градиентного спуска и наискорейшего спуска, которые используются для минимизации квадратичной функции двух переменных:

$$f(x, y) = ax^2 + by^2 + cxy + dx + ey + f$$

В ходе анализа предметной области были выделены следующие ключевые архитектурные сущности:

- **Vector** — математический примитив, который инкапсулирует операции векторной алгебры (*сложение, вычитание, умножение на скаляр, скалярное произведение и вычисление нормы*).
- **QuadraticFunction** — класс, который представляет целевую функцию. Отвечает за вычисление значения функции в точке, градиента, аналитического экстремума и оптимального шага при одномерном поиске.
- **Optimizer** — абстрактный базовый класс, который задаёт единый интерфейс и общие параметры остановки для всех алгоритмов оптимизации.
- **CyclicCoordinateDescent**, **GradientDescent**, **SteepestDescent** — конкретные реализации алгоритмов спуска, которые наследуют базовую логику оптимизатора.

2.6.2 Диаграмма классов

Ниже представлена спроектированная иерархия и взаимосвязи между компонентами объектной модели.

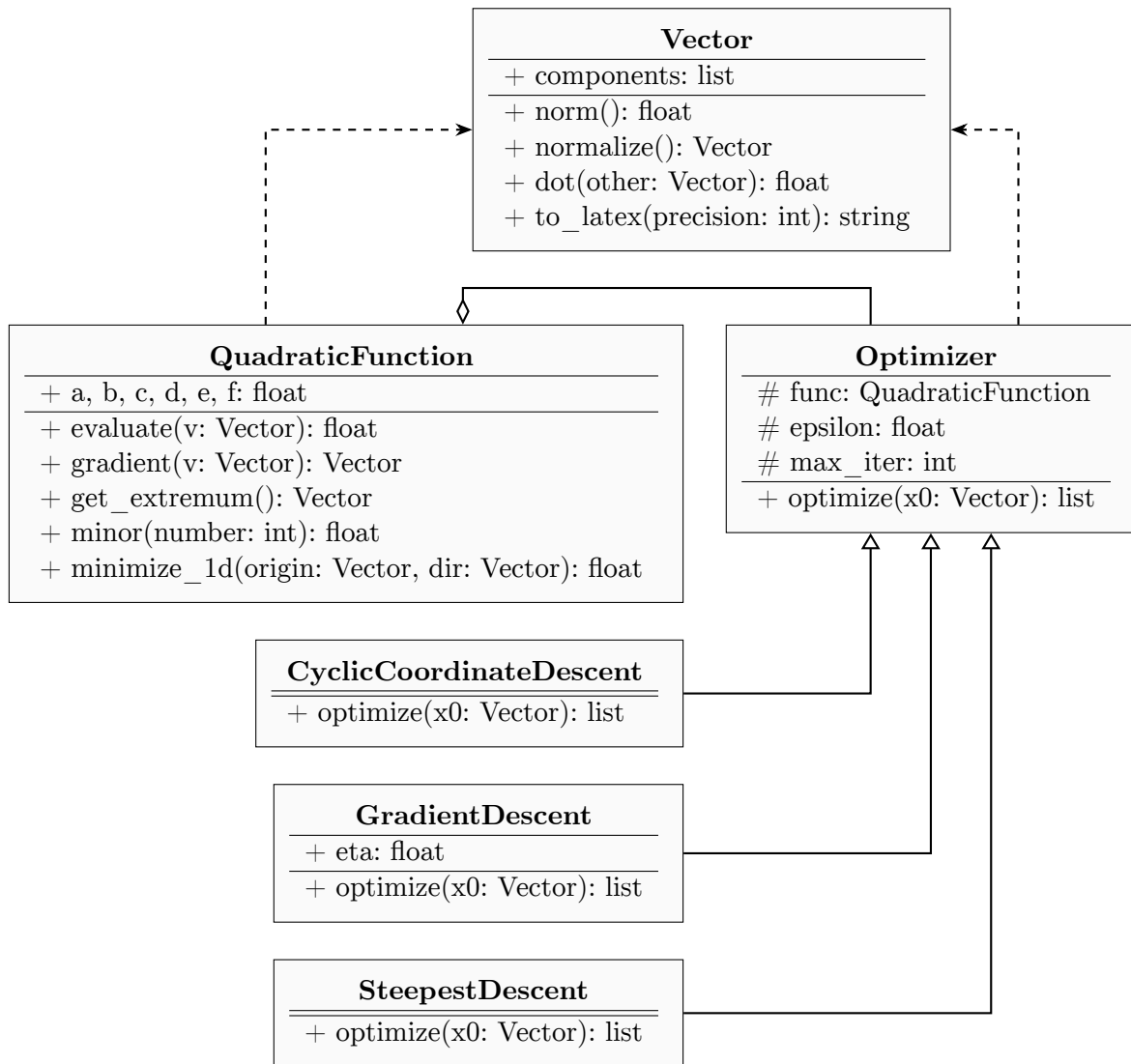


Рис. 7: Диаграмма классов сформулированной объектной модели.

Описание объектных отношений:

- Наследование:** классы `CyclicCoordinateDescent`, `GradientDescent` и `SteepestDescent` расширяют абстрактный класс `Optimizer`. Они переопределяют метод `optimize`, чтобы предоставить стратегию поиска минимума, но делят общее состояние базового класса.
- Агрегация:** абстрактный класс `Optimizer` содержит ссылку на `QuadraticFunction`. Функция передается извне через конструктор, что позволяет оптимизаторам работать с любой конфигурацией квадратичной функции без привязки к её созданию внутри самого метода.
- Зависимость:** классы вычислительных алгоритмов и целевой функции зависят от класса `Vector`. Экземпляры `Vector` передаются в качестве аргументов методов, преобразуются в процессе вычислений и возвращаются как результат работы программы.

3 Блок №3

3.1 Задание

Исследуйте пример из дополнительных материалов к Лекции 6: `UFSACO.ipynb`, данные `samples.pkl`, а также методические указания (*Лекция 6. Комментарии к коду (FS).docx*).

Данный кейс демонстрирует отбор значимых признаков при моделировании содержания меди в продуктах обработки ПАО «Норильский никель» (процесс пенной флотации). Вам необходимо решить аналогичную задачу отбора признаков для собственных данных, используя предложенный программный комплекс.

Порядок выполнения работы:

1. Выберите набор данных для задачи *регрессии* на одном из агрегаторов задач машинного обучения (например, [Kaggle](#), [UCI Machine Learning Repository](#) и др.).

Определите вещественный целевой признак. Приведите в отчете краткое описание предметной области датасета, его структуры и признакового состава, опираясь на информацию с портала и результаты разведочного анализа данных.

2. Используя алгоритм градиентного бустинга (*в режиме «чёрного ящика»*), определите подмножество наиболее значимых признаков для предсказания целевой переменной.
3. С помощью алгоритма ненаправленного муравьиного отбора признаков — UFSACO — выделите значимые признаки *без учета целевой переменной*.
4. Убедитесь, что пересечение двух множеств признаков, полученных методами бустинга и АСО, не является пустым. Мощность пересечения должна составлять **не менее 4 элементов**. Если данное требование не выполняется, поварьируйте гиперпараметры алгоритма муравьиной колонии.

Важно! Допускается использование только той версии алгоритма АСО, которая программно представлена в файле `UFSACO.ipynb`.

Ограничения и требования к датасету:

- Количество признаков: ≥ 20 .
- Количество объектов: ≥ 100 .
- Дата публикации набора данных: не ранее 2010 года.
- Количество отобранных признаков каждым из методов: не менее 7 и не более половины от общего числа признаков в исходном датасете.

- Во избежание дублирования выбранный датасет необходимо предварительно согласовать.

3.2 Описание набора данных

В качестве исследуемого набора данных был выбран датасет, который содержит информацию о наиболее популярных видеоиграх на платформе Steam (*актуальность данных — май 2026 года*[1]). Предметная область охватывает цифровое распространение видеоигр, анализ пользовательских предпочтений и оценку коммерческих характеристик игровых продуктов.

Объект исследования включает в себя 1495 уникальных объектов, каждый из которых описывается 29 признаками. Формат исходных данных — CSV.

3.2.1 Целевой признак

В рамках решаемой задачи регрессии в качестве вещественного целевого признака предлагается взять параметр `metacritic_score`. Это непрерывная величина, которая отражает средневзвешенную оценку игры от профильной прессы по 100-балльной шкале. Задача модели — спрогнозировать данный рейтинг на основе метаданных игры.

3.2.2 Структура и состав признаков

Признаки можно разделить на несколько категорий:

1. Идентификаторы и текстовые данные:

- `app_id` — уникальный идентификатор приложения в системе Steam (*исключается из процесса обучения*);
- `name` — название игры;
- `short_description` — краткое текстовое описание игры для контентного анализа;
- `header_image` — URL-ссылка на главное графическое изображение (баннер) игры на платформе.

2. Количественные признаки:

- `price_usd` — текущая стоимость игры в долларах США;
- `discount_pct` — текущий размер скидки на продукт, выраженный в процентах;
- `recommendations` — общее количество рекомендаций игры от пользователей Steam;
- `estimated_owners` — расчетный интервал количества владельцев копий игры;
- `positive_reviews`, `negative_reviews` — количество положительных и отрицательных отзывов пользователей;
- `avg_playtime_forever` — среднее время в минутах, которое провели пользователи в игре за всё время;

- `avg_playtime_2weeks` — среднее игровое время в минутах, которое провели пользователи в игре за последние две недели;
- `median_playtime` — медианное время, проведенное игроками в игре за всё время;
- `required_age` — минимальное возрастное ограничение, установленное для приобретения и запуска игры;
- `dlc_count` — количество доступных загружаемых дополнений;
- `achievements` — общее количество внутриигровых достижений, доступных для разблокировки;
- `peak_ccu` — пиковое количество одновременных пользователей.

3. Категориальные и множественные признаки:

- `developer, publisher` — разработчик и издатель игры;
- `genres, categories, tags` — списки жанров, технических категорий и пользовательских тегов.

4. Бинарные признаки:

- `is_free` — флаг, который указывает на бесплатную модель распространения (Free-to-Play);
- `coming_soon` — флаг, указывающий, находится ли игра на стадии предзаказа и ожидания релиза;
- `platforms_*` — группа флагов, которые определяют поддержку операционных систем Windows, Mac и Linux.

5. Временные признаки:

- `release_date` — дата релиза игры на платформе.

3.2.3 Разведочный анализ данных

В процессе подготовки данных были изучены их основные особенности и сделаны следующие выводы:

- Распределение `metacritic_score` оказалось смещено в сторону высоких баллов (*медиана около 75–80*). Это логично, так как в популярные игры редко попадают проекты с плохой критикой.
- У части игр нет оценок на Metacritic, поэтому в целевой переменной были пустые значения. Так как предсказывать оценки для игр, по которым нет никаких данных, бессмысленно, я просто удалил такие строки из обучающей выборки.
- Для ряда признаков наблюдались выбросы значений, поэтому из выборки были отсечены объекты с сильно отклоняющимися данными.

3.3 Результаты алгоритмов отбора признаков

Для работы алгоритма *UFSACO* были заданы следующие параметры:

- Первоначальное число признаков: определяется размерностью матрицы `all_scaled`;
- Целевое число признаков: 9;
- Максимальное число циклов: 35;
- Число шагов внутри одного цикла: 2;
- Количество агентов-муравьев в колонии: 6;
- Начальное количество феромона: 0.3;
- Коэффициент испарения феромона: 0.5;
- Вероятность выполнения шага применения: 0.6;
- Параметр важности некоррелированности фичей: 0.5;
- Параметр важности корреляции с целевым признаком: 2.0;
- Малая величина для предотвращения деления на ноль: 10^{-6} .

Сравнительный анализ признаков, отобранных обоими алгоритмами, представлен в таблице 3. Маркером «*» отмечены признаки, попавшие в финальное пересечение множеств.

Признаки Gradient Boosting	Общий	Признаки UFSACO	Общий
achievements		avg_playtime_forever	*
avg_playtime_forever	*	coming_soon	
days_since_release	*	days_since_release	*
median_playtime	*	is_free	
negative_reviews		median_playtime	*
peak_ccu		metacritic_score	
positive_reviews		platforms_mac	
price_usd	*	platforms_win	
recommendations		price_usd	*

Таблица 3: Сравнение множеств значимых признаков

По результатам работы алгоритмов было зафиксировано 4 общих значимых признака. Множество пересечения включает в себя следующие параметры:

```
['median_playtime', 'price_usd', 'days_since_release',  
 'avg_playtime_forever']
```

Тот факт, что эти признаки были отобраны как градиентным бустингом, так и UFSACO, как минимум показывает их стабильность:

- `price_usd` определяет рыночное позиционирование игрового продукта;
- `avg_playtime_forever` и `median_playtime` отражают реальную вовлеченность аудитории;
- `days_since_release` выступает ключевым временным предиктором, инкапсулирующим «возраст» игры на платформе Steam.

4 Источники литературы

1. *Patel R.* Steam Top 1495 Games Dataset. — 05.2026. — URL: <https://www.kaggle.com/datasets/patelris/steam-top-1495-games-dataset> (дата обр. 30.05.2026) ; Version 9.